

WIPRO TRAINING
CAPSTONE PROJECT

TEST PLAN

Guru99 Bank Automation Testing Framework

Document Type	Test Plan
Project Name	Guru99 Bank Automation
Version	1.0
Prepared By	QA Automation Engineer
Date	June 2026
Status	FINAL

1. Objective

The objective of this project is to design, develop, and execute an automated testing framework for the Guru99 Bank web application. The framework is built using industry-standard tools and follows best practices to ensure maintainability, scalability, and reusability.

The automation framework aims to validate the following critical banking functionalities:

Module	Coverage	Priority
Login / Logout	Valid, Invalid, Empty credentials	Critical
Customer Management	Add, Edit, Delete Customer	High
Account Management	New, Edit, Delete Account	High
Deposit	Valid, Invalid, Boundary values	High
Withdrawal	Valid, Insufficient funds, Invalid input	High
Fund Transfer	Valid, Same account, Insufficient balance	High
Mini Statement	Balance enquiry, transaction history	Medium
Change Password	Valid, Invalid, Mismatch passwords	Medium

2. Scope

2.1 In Scope

The following modules and test scenarios are included within the automation scope:

Module	Sub-Module / Scenario	Test Types
Login Module	Valid Login, Invalid Credentials, Empty Fields	Positive, Negative
Customer Module	Add Customer, Edit Customer, Delete Customer	Positive, Negative, Data-Driven
Account Module	New Account, Edit Account, Delete Account	Positive, Negative
Deposit Module	Valid Deposit, Invalid Amount, Empty Fields	Positive, Negative, Boundary

Withdrawal Module	Valid Withdrawal, Low Balance, Invalid Input	Positive, Negative, Boundary
Fund Transfer	Valid Transfer, Same Account, Low Balance	Positive, Negative
Security Module	Logout	Positive, Negative

2.2 Out of Scope

The following areas are explicitly excluded from the automation testing effort:

Area	Reason for Exclusion	Status
Database Testing	No direct DB access available on demo site	Excluded
API Testing	Application does not expose public APIs	Excluded
Performance Testing	Out of functional scope for this project	Excluded
Security Pen Testing	Requires specialized tools outside project scope	Excluded
Mobile Testing	Web-only application; mobile not in scope	Excluded

3. Test Strategy

The test strategy defines how testing will be approached, executed, and validated throughout the project lifecycle.

Attribute	Details
Automation Type	Functional UI Automation Testing
Framework Type	Hybrid Framework — Page Object Model (POM) + Data-Driven Testing
Testing Approach	Black Box Testing — functionality tested without knowledge of internal code
Test Execution	Sequential execution via TestNG XML suite; parallel execution ready
Test Data Management	Externalized in Excel (.xlsx) files using Apache POI; no hardcoded test data
Browser Coverage	Google Chrome (primary); configurable via testng.xml parameter
Reporting	Extent Reports for HTML dashboards; TestNG built-in reports
CI/CD Integration	Maven + Docker containerized execution; Jenkins pipeline ready
Assertions	Hard Assert for critical validations; Soft Assert for non-blocking checks

3.1 Framework Architecture

The framework follows a layered architecture to ensure clean separation of concerns:

Layer	Package	Responsibility
Base Layer	base.*	WebDriver lifecycle, browser setup, login setup
Page Layer	pages.*	Page Object classes, element locators, page actions
Test Layer	tests.*	TestNG test classes, DataProvider, assertions
Utilities Layer	utilities.*	ExcelUtil, WaitUtil, ReportUtil, DriverFactory
Test Data	src/test/resources/testdata/	Excel files for all data-driven test cases

4. Tools & Technologies

The following tools and technologies are used to build and execute the automation framework:

Tool / Technology	Version	Purpose
Java	JDK 21	Core programming language for the framework
Selenium WebDriver	4.x	Browser automation and UI interaction
TestNG	7.x	Test execution, grouping, data providers, and assertions
Maven	3.9.x	Build management, dependency handling, test execution via Surefire plugin
Apache POI	5.x	Reading Excel (.xlsx) files for data-driven testing
Extent Reports	5.x	HTML test execution reports with pass/fail/skip dashboards
Log4j2	2.x	Structured logging of test steps and framework events
GitHub	N/A	Source code version control and collaboration
Docker	Latest	Containerized headless test execution environment
Jenkins	LTS	CI/CD pipeline for automated test triggering and reporting

Eclipse IDE	2026.x	Development environment for framework authoring and debugging
-------------	--------	---

5. Test Environment

Parameter	Details
Application Under Test	Guru99 Bank Demo Application
Application URL	https://demo.guru99.com/V4/
Browser	Google Chrome (Latest Stable)
Operating System	Windows 11 (local) / Ubuntu (Docker)
Java Version	JDK 21 (eclipse-temurin)
Execution Mode	Local (Eclipse) / Headless (Docker Container)

6. Entry & Exit Criteria

6.1 Entry Criteria

The following conditions must be met before test execution begins:

#	Criteria	Verification Method
1	Application URL is accessible and stable	Open browser and confirm login page loads
2	Valid test credentials are available	Login manually with credentials before test run
3	Test data Excel files are populated and placed in testdata/ folder	Verify files exist in src/test/resources/testdata/
4	Automation framework is built successfully (mvn clean install)	No build errors in Maven output
5	ChromeDriver version matches installed Chrome browser	Run driver version check command
6	testng.xml is configured with correct suite and parameters	Review XML before execution

6.2 Exit Criteria

Test execution is considered complete when the following conditions are satisfied:

#	Criteria	Acceptance Threshold
1	All critical and high-priority test cases executed	100% execution rate for P1/P2 tests
2	No open Critical or High severity defects	Zero unresolved critical/high bugs
3	Test pass rate meets minimum threshold	90% or above pass rate
4	Extent Report generated and reviewed	Report available in target/extent-reports/
5	All test results documented in test execution report	Excel/HTML report with PASS/FAIL status per test
6	Defect report submitted for all failed tests	Each failure has a corresponding defect entry

7. Risks & Mitigation

The following risks have been identified and corresponding mitigation strategies have been planned:

#	Risk	Likelihood	Impact	Mitigation Strategy
1	Demo site downtime or instability	Medium	High	Schedule runs during off-peak hours; add retry logic in framework
2	Dynamic test data (account IDs change per session)	High	High	Fetch account IDs dynamically after customer creation; avoid hardcoded IDs
3	Browser/ChromeDriver version mismatch	Medium	Medium	Use WebDriverManager for auto driver management
4	Test flakiness due to page load timing	Medium	Medium	Replace implicit waits with explicit WebDriverWait via WaitUtil
5	Locator changes due to site updates	Low	High	Centralize all locators in Page Object classes for easy updates
6	Docker/CI environment differences	Low	Medium	Use headless Chrome with --no-sandbox and --disable-dev-shm-usage flags

8. Test Schedule

#	Activity	Start	End
1	Framework Design & Setup	Week 1	Week 1

2	Page Object Development	Week 1	Week 1
3	Test Case Development	Week 1	Week 1
4	Test Data Preparation (Excel)	Week 1	Week 1
5	Test Execution & Debugging	Week 1	Week 1
6	Docker & CI/CD Integration	Week 1	Week 1
7	Report Generation & Review	Week 1	Week 1
8	Documentation & Handover	Week 1	Week 1

9. Deliverables

#	Deliverable	Format	Status
1	Automation Framework Source Code	Java/Maven Project (GitHub)	Completed
2	Test Plan Document	Word Document (.docx)	Completed
3	Test Cases Document	Excel / Word	Completed
4	Test Data Files	.xlsx (Apache POI)	Completed
5	Test Execution Report (Extent)	HTML Dashboard	Completed
6	Defect Report	Excel / Word	Completed
7	Dockerfile & CI Configuration	Dockerfile / Jenkinsfile	Completed
8	README / Setup Guide	Markdown (.md)	Completed

10. Document Approvals

Role	Name	Signature	Date
Prepared By	QA Automation Engineer	Shahid raza	June 2026
Reviewed By	Trainer		
Approved By	Wipro Official		